一. Web开发模式

目前主流的Web开发模式有两种,分别是:

- 基于**服务端渲染**的传统 Web开发模式
  - 服务端渲染的概念:**服务器发送给客户端的HTML页面,是在服务器通过字符串的拼接,动态生成的。**因此,客户端不需要使用Ajax这样的技术额外请求页面的数据。代码示例如下:

```
app.get('/index.html', (req, res) => {
 // 1.要渲染的数据
 const user = { name: 'zs', age: 20 }
 //2. 服务器端通过字符串的拼接,动态生成 HTML 内容
 const html=‘<h1>姓名:${user.name},年龄:${user.age}</h1>'
 //3.把生成好的页面内容响应给客户端。因此,客户端拿到的是带有真实数据的 HTML页面
 res. send(html)
})
```

  - 优点:
    - 前端耗时少。因为服务器端负责动态生成HTML内容,浏览器只需要直接渲染页面即可。尤其是移动端,更省电。
    - **有利于SEO。因为服务器端响应的是完整的HTML页面内容**,所以爬虫更容易爬取获得信息,更有利于SEO。
      - SEO（搜索引擎优化）一句话：**让百度 / 谷歌爬虫能顺利抓取、读懂你的网页，提高网页在【自然搜索结果】里的排名，拿到免费流量。**
  - 缺点:
    - 占用服务器端资源。即服务器端完成HTML页面内容的拼接,如果请求较多,会对服务器造成一定的访问压力。
    - 不利于前后端分离,开发效率低。使用服务器端渲染,则无法进行分工合作,尤其对于前端复杂度高的项目,不利于项目高效开发。
- 基于**前后端分离**的新型 Web 开发模式
  - 前后端分离的概念:前后端分离的开发模式,依赖于Ajax技术的广泛应用。简而言之,前后端分离的Web开发模式,**就是后端只负责提供API接口,前端使用Ajax调用接口的开发模式**。
  - 优点
    - 开发体验好。前端专注于UI页面的开发,后端专注于api的开发,且前端有更多的选择性。
    - 用户体验好。Ajax技术的广泛应用,极大的提高了用户的体验,可以轻松实现页面的局部刷新。
    - 减轻了服务器端的渲染压力。因为页面最终是在每个用户的浏览器中生成的。
  - 缺点
    - 不利于SEO。因为完整的HTML页面需要在客户端动态拼接完成,所以爬虫对无法爬取页面的有效信息。(解决方案:利用Vue、React等前端框架的SSR(server side render)技术能够很好的解决SEO问题!)

选择开发模式要结合实际的业务场景:

- 比如企业级网站,主要功能是展示而没有复杂的交互,并且需要良好的SEO,则这时我们就需要使用服务器端渲

- 而类似后台管理项目,交互性比较强,不需要考虑SEO,那么就可以使用前后端分离的开发模式。
- 另外,具体使用何种开发模式并不是绝对的,为了同时兼顾了首页的渲染速度和前后端分离的开发效率,一些网站采用了首屏服务器端渲染+其他页面前后端分离的开发模式。

二. 身份认证

1.什么是身份认证

身份认证(Authentication)又称"身份验证"、"鉴权",是指**通过一定的手段,完成对用户身份的确认。**

- 日常生活中的身份认证随处可见,例如:高铁的验票乘车,手机的密码或指纹解锁,支付宝或微信的支付密码等。
- 在Web开发中,也涉及到用户身份的认证,例如:**各大网站的手机验证码登录、邮箱密码登录、二维码登录等。**

2. 不同开发模式下的身份认证

对于服务端渲染和前后端分离这两种开发模式来说,分别有着不同的身份认证方案:

- **服务端渲染推荐使用Session 认证机制**
- **前后端分离推荐使用JWT认证机制**

三. Session 认证机制

1. HTTP协议的无状态性

HTTP协议的无状态性,指的是**客户端的每次HTTP请求都是独立的,连续多个请求之间没有直接的关系**,服务器不会主动保留每次 HTTP请求的状态。

- 在web开发中使用Cookie来突破HTTP无状态的限制

2. Cookie

Cookie 是**存储在用户浏览器**中的一段不超过4KB的**字符串**。它由一个名称(Name)、一个值(Value)和其它几个用 于控制 Cookie **有效期**、安全性、使用范围的可选属性组成。

不同域名下的Cookie各自独立,每当客户端发起请求时,会自动把**当前域名下所有未过期的Cookie**一同发送到服务器。

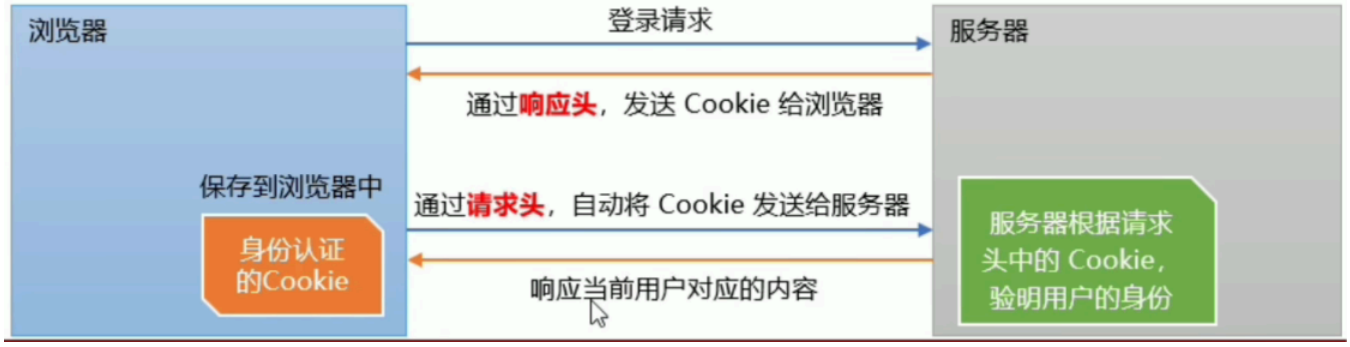
Cookie的几大特性:

- 自动发送、域名独立、过期时限、4KB限制

3.Cookie在身份认证中的作用

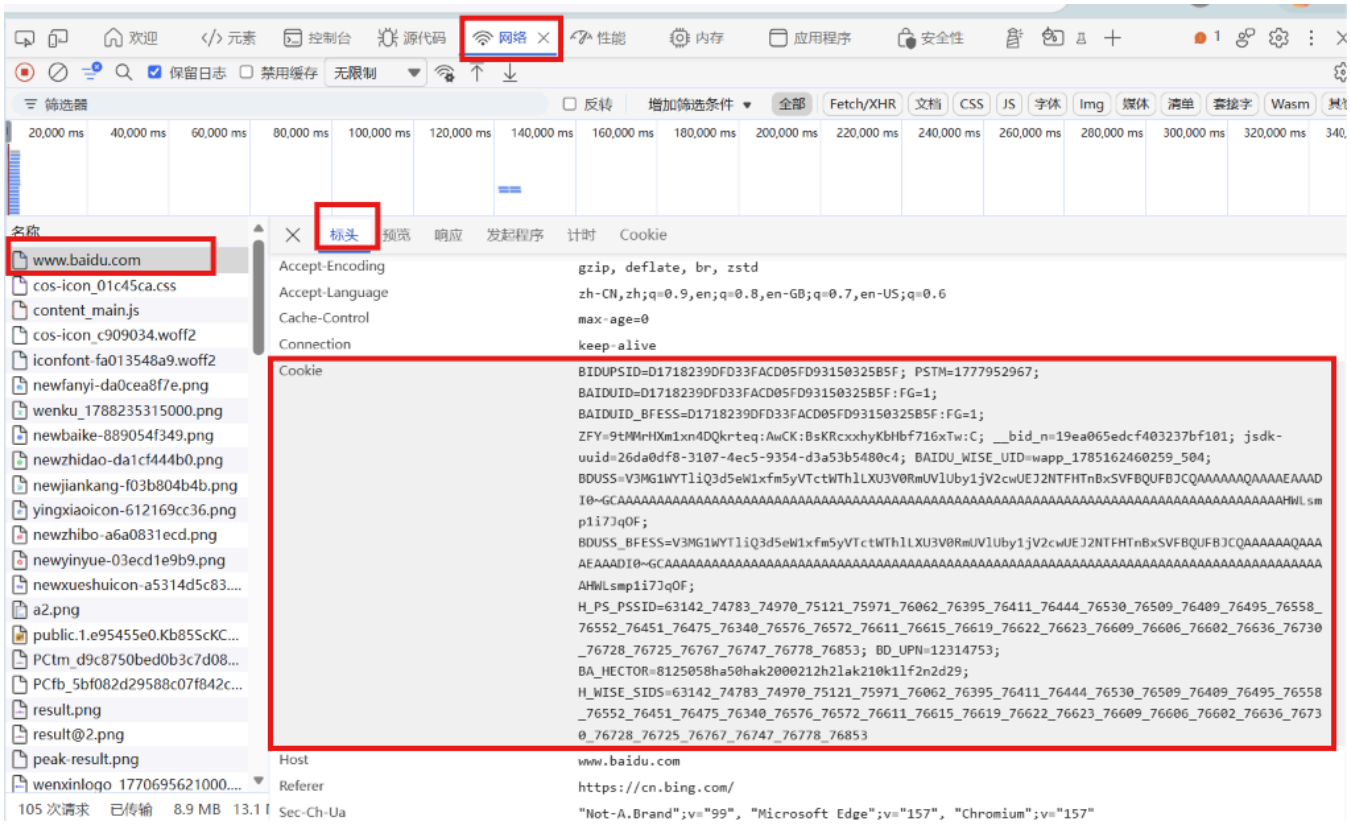
客户端**第一次请求服务器**的时候,服务器**通过响应头的形式**,向客户端发送一个身份认证的Cookie,客户端会自动将Cookie保存在浏览器中。

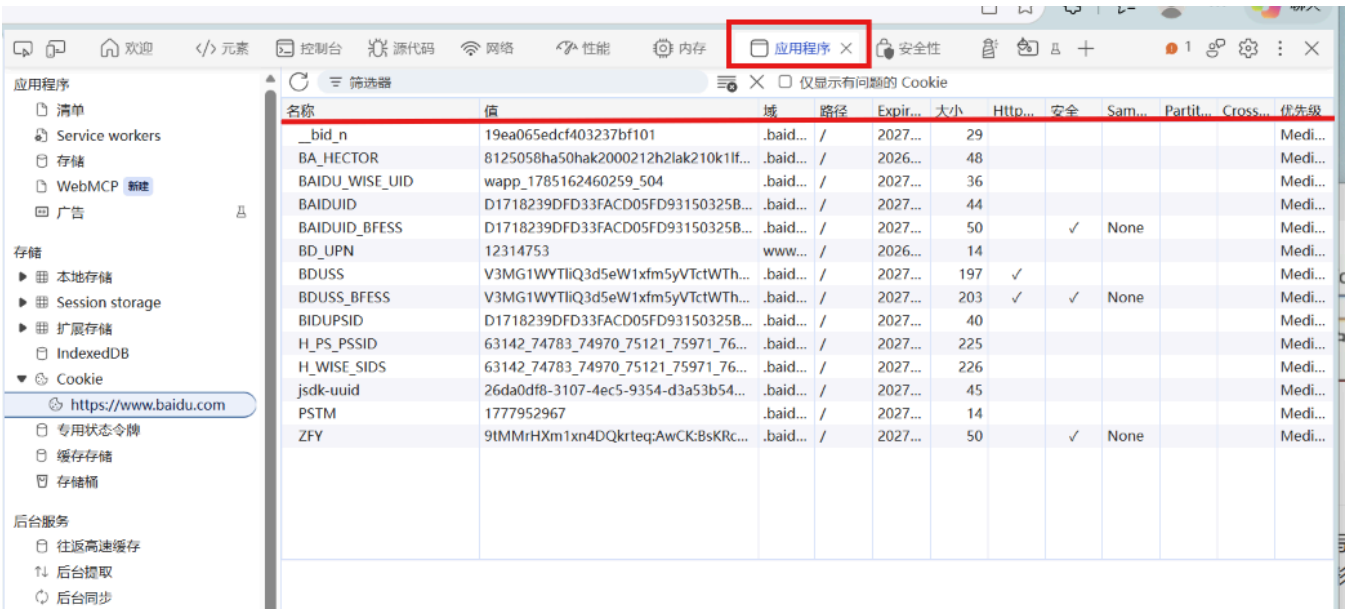
随后,当客户端浏览器每次请求服务器的时候,浏览器会**自动**将身份认证相关的Cookie,**通过请求头的形式**发送给服务器,服务器即可**验明**客户端的身份。



4. Cookie 不具备安全性

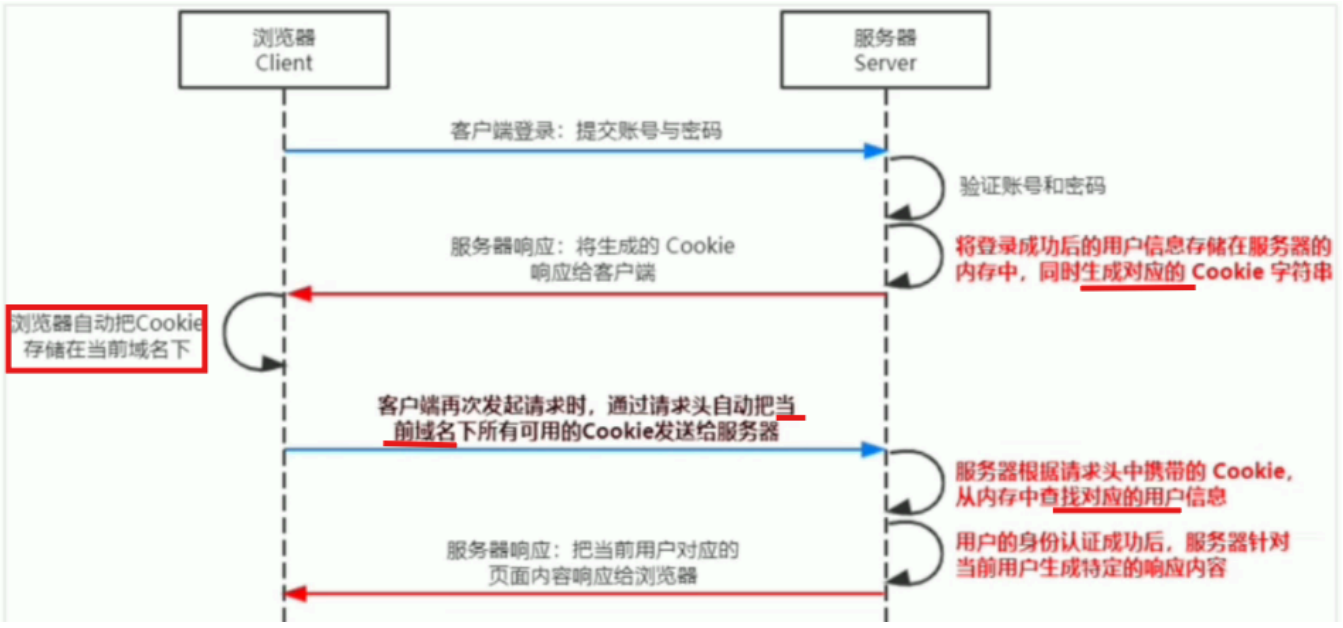
由于Cookie是存储在浏览器中的,而且浏览器也提供了读写Cookie的API（Cookie 存在浏览器，开发者工具（Application）里直接就能看到、改值）,因此Cookie很容易被伪造,不具有安全性。因此不建议服务器将重要的隐私数据,通过Cookie的形式发送给浏览器。


欢迎
元素
控制台
源代码
网络
性能
内存
应用程序
安全性
保留日志
禁用缓存
无限制
筛选器
反转
增加筛选条件
全部
Fetch/XHR
文档
CSS
JS
字体
Img
媒体
清单
套接字
Wasm
20,000 ms
40,000 ms
60,000 ms
80,000 ms
100,000 ms
120,000 ms
140,000 ms
160,000 ms
180,000 ms
200,000 ms
220,000 ms
240,000 ms
260,000 ms
280,000 ms
300,000 ms
320,000 ms
名称
www.baidu.com
cos-icon_01c45ca.css
content_main.js
cos-icon_c909034.woff2
iconfont-fa013548a9.woff2
newfanyi-da0cea8f7e.png
wenku_1788235315000.png
newbaike-889054f349.png
newzhidao-da1cf444b0.png
newjiankang-f03b804b4b.png
yingxiaoicon-612169cc36.png
newzhibo-a6a0831ecd.png
newyinyue-03ecd1e9b9.png
newxueshuicon-a5314d5c83....
a2.png
public.1.e95455e0.Kb85ScKC...
PCtm_d9c8750bed0b3c7d08...
PCfb_5bf082d29588c07f842c...
result.png
result@2.png
peak-result.png
wenxinlogo_1770695621000....
105 次请求
已传输
8.9 MB
13.1 I
标头
预览
响应
发起程序
计时
Cookie
Accept-Encoding
gzip, deflate, br, zstd
Accept-Language
zh-CN,zh;q=0.9,en;q=0.8,en-GB;q=0.7,en-US;q=0.6
Cache-Control
max-age=0
Connection
keep-alive
Cookie
BIDUPSID=D1718239DFD33FACD05FD93150325B5F; PSTM=1777952967;
BAIDUID=D1718239DFD33FACD05FD93150325B5F:FG=1;
BAIDUID_BFESS=D1718239DFD33FACD05FD93150325B5F:FG=1;
ZFY=9tMMrHXm1xn4DQkrteq:AwCK:BsKRcxxhyKbHbf716xTw:C; __bid_n=19ea065edcf403237bf101; jsdk-
uuid=26da0df8-3107-4ec5-9354-d3a53b5480c4; BAIDU_WISE_UID=wapp_1785162460259_504;
BDUSS=V3MG1WYTliQ3d5eW1xfm5yVTctWThlLXU3V0RmUVlUby1jV2cwUEJ2NTFHTnBxSVFBQUFBJCQAAAAAAQAAAAEAAAD
I0~GCAAAAAA



注意:千万不要使用Cookie存储重要且隐私的数据!比如用户的身份信息、密码等。

5. Session的工作原理



6.在Express 中使用 Session认证

- 安装：在Express项目中,只需要安装**express-session**中间件,即可在项目中使用Session认证:

```
npm i express-session
```

- 配置：express-session 中间件安装成功后,需要通过**app.use()**来注册session中间件

```
// 1. 导入 session 中间件
var session = require( 'express-session')

// 2. 配置 Session 中间件
app.use(session({
    secret:'keyboard cat',  //secret 属性的值可以为任意字符串
    resave: false,          // 固定写法
    saveUninitialized: true //固定写法
}))
```

- 向session中存数据：当 express-session中间件配置成功后,即可通过**req.session** 来访问和使用session对象,从而存储用户的关键信息

```
app.post('/api/login', (req, res) => {
    //判断用户提交的登录信息是否正确
    if (req.body.username !== 'admin' || req.body.password !== '000000') {
        return res.send({ status: 1,msg:'登录失败'})
    }

    req.session.user = req.body //将用户的信息,存储到 Session 中
    req.session.islogin=true //将用户的登录状态,存储到Session 中

    res.send({ status: 0, msg:'登录成功'})
})
```

- 从session中取数据：可以直接从 **req.session** 对象上获取之前存储的数据

```
// 获取用户姓名的接口
app.get('/api/username', (req, res) => {
    // 判断用户是否登录
    if (!req.session.islogin) {
        return res.send({ status: 1, msg: 'fail' })
    )
    res. send({ status: 0, msg: 'success', username: req.session.user.username })
})
```

- 清空session：调用 **req.session.destroy()**函数,即可清空服务器保存的session信息。

```
// 退出登录的接口
app.post('/api/logout', (req, res) => {
    // 清空当前客户端对应的 session 信息
    req. session.destroy()
    res. send({
        status: 0,
        msg:‘退出登录成功'
    })
})
```

四. JWT 认证机制

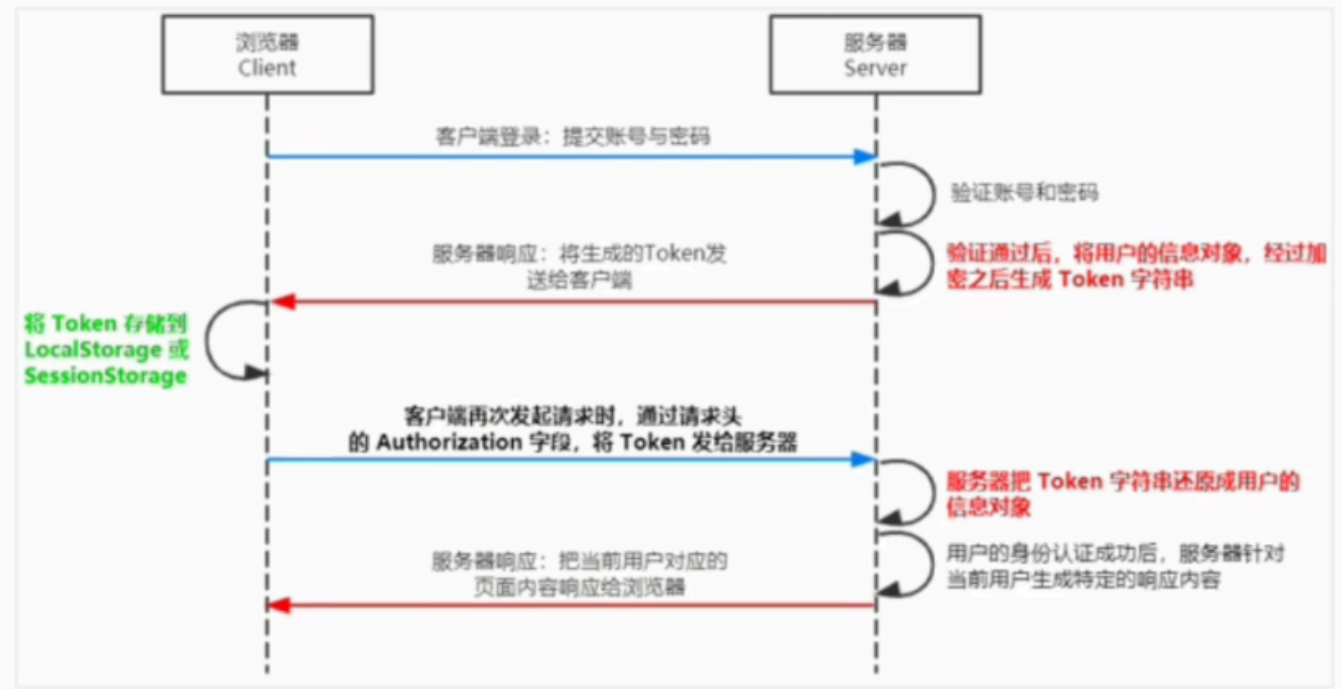
1.了解Session认证的局限性

Session认证机制需要配合Cookie才能实现。**由于Cookie默认不支持跨域访问**,所以,当涉及到前端跨域请求后端接 口的时候,需要做很多额外的配置,才能实现跨域Session认证。

- 当前端请求后端接口**不存在跨域问题**的时候,推荐使用**Session身份认证**机制。
- 当**前端需要跨域请求后端接口**的时候,不推荐使用Session身份认证机制,推荐使用**JWT认证**机制。

2.JWT工作原理

JWT(英文全称:JSON Web Token)是目前最流行的**跨域认证解决方案。**



总结:**用户的信息通过Token字符串的形式,保存在客户端浏览器中。服务器通过还原Token字符串的形式来认证用户的身份。**

在JWT中用户信息保存在浏览器中（用户端）—— 用户信息以Token字符串形式呈现

在Session中用户信息保存在服务器端

3.JWT的组成部分

JWT通常由三部分组成,分别是**Header(头部)、Payload(有效荷载)、Signature(签名)。**

三者之间使用英文的“.”分隔,格式如下:`Header .Payload. Signature`

- **Payload部分才是真正的用户信息,它是用户信息经过加密之后生成的字符串。**
- Header和 Signature是安全性相关的部分,只是为了保证Token的安全性。

4.JWT的使用方式

客户端收到服务器返回的JWT之后,通常会将它储存在**localStorage或sessionStorage**中。

此后,客户端每次与服务器通信,都要带上这个JWT的字符串,从而进行身份认证。**推荐的做法是把JWT放在HTTP 请求头的Authorization字段中**,格式如下:

```
Authorization: Bearer <token>
```

5. 在Express 中使用JWT

- 安装包：jsonwebtoken 用于生成JWT字符串 express-jwt 用于将JWT 字符串解析还原成JSON 对象

```
npm install jsonwebtoken express-jwt
```

- 导包：

```
// 1.导入用于生成 JWT 字符串的包
const jwt = require( 'jsonwebtoken')
// 2. 导入用于将客户端发送过来的 JWT 字符串,解析还原成 JSON 对象的包
const expressJWT = require('express-jwt')
```

- 定义secret密钥：为了保证JWT字符串的安全性,防止JWT字符串在网络传输过程中被别人破解,我们需要专门定义一个**用于加密和解密的secret密钥**
    - 当**生成JWT字符串的时候,需要使用secret密钥对用户的信息进行加密,最终得到加密好的JWT字符串**
    - 当**把JWT字符串解析还原成JSON对象的时候,需要使用secret密钥进行解密**

    ```
    // 3. secret 密钥的本质:就是一个字符串
    const secretKey = 'sacsdcadca'
    ```

- 在登录成功后生成JWT字符串

  调用**jsonwebtoken包提供的sign()**方法,将用户的信息加密成JWT字符串,响应给客户端:

```
//登录接口
app.post('/api/login', function(req, res) {
// ... 省略登录失败情况下的代码
//用户登录成功之后,生成 JWT 字符串,通过 token 属性响应给客户端
    res. send( {
        status: 200,
        message:'登录成功!',
        // 调用 jwt,sign()生成JWT 字符串,三个参数分别是:用户信息对象、加密密钥、配置对象
        token: jwt.sign({ username: userinfo.username }, secretKey, {expiresIn: '30s' })
    })
})

// expiresIn: '30s'  从生成的那一刻开始，30s内是有效的
```

- 将JWT字符串还原为JSON 对象

  家户端每次在访问那些有权限接口的时候,都需要主动通过**请求头中的Authorization字段**,将Token字符串发 送到服务器进行身份认证。

  此时,服务器可以通过**express-jw**t这个中间件,自动将客户端发送过来的Token解析还原成JSON对象:

  ```
  // 使用 app.use()来注册中间件
  // expressJWT({ secret:secretKey})就是用来解析 Token 的中间件
  // .unless({ path:[/^\/api\//]})用来指定哪些接口不需要访问权限
  app.use(expressJWT({ secret: secretKey }).unless({ path:[/^\/api\//] }))
  ```

- 使用 req.user 获取用户信息

  当express-jwt这个中间件配置成功之后,即可在那些有权限的接口中,使用**req.user**对象,来访问从JWT字符串 中解析出来的用户信息了,示例代码如下: